R: limpieza, manipulación y exploración de datos

Cheatsheet de expresiones regulares

Jonatán Perren

26 de marzo de 2026

¿Qué es una expresión regular?

Un mini-lenguaje para describir patrones de texto

Una expresión regular (regex) es una secuencia de caracteres que define un patrón de búsqueda. En R, se usan principalmente con las funciones de `stringr` y con funciones base como `grep()`, `gsub()`, etc.

¿Para qué se usan en el análisis de datos?

- Extraer información de texto no estructurado (precios, fechas, códigos).
- Limpiar datos cargados con formatos inconsistentes.
- Validar el formato de observaciones (ej. CUIT, patentes).
- Filtrar filas que contengan ciertos patrones textuales como emails.

En R, las barras invertidas se duplican. Para representar `\d` en una regex, escribimos `"\\d"` en el código.

Caracteres literales

La forma más simple de regex: buscar un texto exacto.

```r
library(stringr)

x <- c("Santa Fe", "San Luis", "Salta", "Mendoza")
```

```
> str_detect(x, "San")
[1]  TRUE  TRUE FALSE FALSE
>
> str_extract(x, "San")
[1] "San" "San" NA    NA
```

Las regex son **sensibles a mayúsculas** por defecto. `"san"` no matchea `"San"`. Usá `regex("san", ignore_case = TRUE)` si necesitás ignorar la distinción.

Metacaracteres

Estos caracteres tienen un significado especial en regex. Si querés buscar el carácter literal, hay que escaparlos con `\\`.

Metacarácter	Significado
`.`	Cualquier carácter (excepto salto de línea)
`^`	Inicio de la cadena
`$`	Final de la cadena
`*`	0 o más repeticiones
`+`	1 o más repeticiones
`?`	0 o 1 repetición (opcional)
`{}`	Cuantificador exacto
`[]`	Clase de caracteres
`()`	Grupo de captura
`|`	Alternancia (OR)
`\`	Escape

Escapar metacaracteres

Para buscar un metacarácter de forma literal, se lo escapa con `\\`. En R, como las barras se duplican, la tarea se hace así

```
# Buscar un punto literal
str_detect("3.14", "\\.")
```

```
[1] TRUE
```

```
# Sin escapar, el punto matchea cualquier carácter
str_detect("3X14", ".")
```

```
[1] TRUE
```

Este último TRUE es por el `"3"`, o `"X"` o `"1"` o `"4"`

```
# Buscar el signo $
str_extract("Precio: $150", "\\$[0-9]+")
```

```
[1] "$150"
```

```
# Buscar paréntesis literales
str_extract("Juan (25)", "\\(.+\\)")
```

```
[1] "(25)"
```

Alternativamente, `fixed()` evita interpretar metacaracteres:

```
str_detect("3.14", fixed("."))
```

```
[1] TRUE
```

El punto: Comodín universal

El caracter `.` coincide con **cualquier carácter** excepto salto de línea.

```r
x <- c("gato", "grito", "goto", "gusto")

str_detect(x, "g.to")
```

```
[1]  TRUE FALSE  TRUE FALSE
```

```r
str_extract(x, "g.to")
```

```
[1] "gato" NA     "goto" NA
```

```r
# Dos puntos: dos caracteres cualesquiera
str_detect(x, "g..to")
```

```
[1] FALSE  TRUE FALSE  TRUE
```

```r
str_extract(x, "g..to")
```

```
[1] NA      "grito" NA      "gusto"
```

Anclas: ^ y $

Las anclas no coinciden con caracteres, sino con **posiciones**.

^ marca el inicio de la cadena

```r
x <- c("ingreso neto", "total ingreso", "ingreso")

str_detect(x, "^ingreso")
```

```
[1]  TRUE FALSE  TRUE
```

$ marca el final de la cadena

```r
str_detect(x, "ingreso$")
```

```
[1] FALSE  TRUE  TRUE
```

Combinados: cadena exacta

```r
# Solo matchea "ingreso" exactamente
str_detect(x, "^ingreso$")
```

```
[1] FALSE FALSE  TRUE
```

Clases de caracteres: [ ]

Los corchetes definen un **conjunto** de caracteres posibles para una posición.

```r
x <- c("gato", "geto", "gito", "goto", "gusto")

# Matchea 'g' + vocal + 'to'
str_extract(x, "g[aeiou]to")
```

```
[1] "gato" "geto" "gito" "goto" NA
```

Rangos dentro de clases

Si queremos matchear cualquier letra, no es necesario escribir una cadena con cada una. Para eso hay rangos.

- `"A-Z"`: Un caracter en mayúsculas entre la `"A"` y la `"Z"`.
- `"a-z"`: Un caracter en minúsculas entre la `"a"` y la `"z"`.
- `"0-9"`: Un dígito numérico.

```r
x <- c("A1", "b2", "C3", "d4", "5E")

str_extract(x, "[A-Z][0-9]") # Una mayúscula + un dígito
```

```
[1] "A1" NA   "C3" NA   NA
```

```r
str_extract(x, "[a-z][0-9]") # Una minúscula + un dígito
```

```
[1] NA   "b2" NA   "d4" NA
```

```r
str_extract(x, "[A-Za-z][0-9]") # Cualquier letra (una vez) + un dígito
```

```
[1] "A1" "b2" "C3" "d4" "5E"
```

Clases de caracteres: Negación y combinaciones

Negación con ^ dentro de [ ]

Cuando ^ aparece **dentro** de corchetes, niega la clase.

```r
x <- c("abc", "a1c", "a c", "a.c")

# Cualquier carácter que NO sea letra
str_extract(x, "a[^a-zA-Z]c")
```

```
[1] NA    "a1c" "a c" "a.c"
```

Múltiples rangos y caracteres

```r
# Dígitos o punto (para parsear decimales)
str_extract_all("Precio: 1.250,50",
                "[0-9.,]+")
```

```
[[1]]
[1] "1.250,50"
```

Dentro de `"[]"`, el punto y la mayoría de metacaracteres son literales; no necesitan escape. Excepciones: `"]"` y `"\"`.

Atajos de clase

Son abreviaciones para clases de caracteres frecuentes.

Atajo	Descripción	Equivale a
`\d`	Dígito	`[0-9]`
`\D`	NO dígito	`[^0-9]`
`\w`	Carácter de «palabra»	`[A-Za-z0-9_]`
`\W`	NO carácter de «palabra»	`[^A-Za-z0-9_]`
`\s`	Espacio en blanco	`[ \t\n\r]`
`\S`	NO espacio en blanco	`[^ \t\n\r]`
`\b`	Límite de palabra	`(posición)`

Recordá que en R se duplica la barra: `"\\d"`, `"\\w"`, etc.

Atajos de clase en acción

```r
x <- "Total: $1.250 (año 2025)"

# Extraer todos los números
str_extract_all(x, "\\d+") #Recordá que el + es un cuantificador.
```

```
[[1]]
[1] "1"    "250"  "2025"
```

```r
# Extraer palabras
str_extract_all(x, "\\w+")
```

```
[[1]]
[1] "Total" "1"     "250"   "año"   "2025"
```

```r
# Extraer todo lo que NO sea espacio
str_extract_all(x, "\\S+")
```

```
[[1]]
[1] "Total:"  "$1.250" "(año"   "2025)"
```

Límites de palabra: \b

`"\\b"` coincide con una **posición** entre un carácter de palabra y uno que no lo es. No consume caracteres.

```r
x <- "El ingreso total y los ingresos parciales"

# Sin límite: matchea "ingreso" dentro de "ingresos"
str_extract_all(x, "ingreso")
```

```
[[1]]
[1] "ingreso" "ingreso"
```

```r
# Con límite: solo la palabra completa "ingreso"
str_extract_all(x, "\\bingreso\\b")
```

```
[[1]]
[1] "ingreso"
```

```r
# Palabras que empiezan con "in"
str_extract_all(x, "\\bin\\w+")
```

```
[[1]]
[1] "ingreso"  "ingresos"
```

Cuantificadores

Controlan **cuántas veces** se repite el elemento que los precede.

Cuantificador	Repeticiones	Ejemplo
`*`	0 o más	`ab*c` matchea ac, abc, abbc...
`+`	1 o más	`ab+c` matchea abc, abbc...
`?`	0 o 1	`colou?r` matchea color, colour
`{n}`	Exactamente n	`\d{4}` matchea 2025
`{n,}`	n o más	`\d{2,}` matchea 25, 250...
`{n,m}`	Entre n y m	`\d{2,4}` matchea 25, 250, 2025

Los cuantificadores son **codiciosos** (greedy) por defecto: capturan la mayor cantidad posible. Para hacerlos perezosos (lazy), se agrega `?` después (ej. `.+?`).

Cuantificadores en acción

```r
# * (0 o más): CUIT puede tener o no guiones
cuits <- c("20-12345678-9", "20123456789")
str_detect(cuits, "\\d{2}-?\\d{8}-?\\d{1}")
```

```
[1] TRUE TRUE
```

```r
# {n}: Años de exactamente 4 dígitos
x <- "En 2025 se registraron 15 casos."
str_extract_all(x, "\\b\\d{4}\\b")
```

```
[[1]]
[1] "2025"
```

```r
# + (1 o más): Extraer secuencias de dígitos
str_extract_all("IPC: 5.2, PBI: 150.3",
                "\\d+\\.?\\d*")
```

```
[[1]]
[1] "5.2"   "150.3"
```

```r
# {n,m}: Códigos postales de 4 o 5 dígitos
str_extract_all("CP: 3000, otro: 12345",
                "\\b\\d{4,5}\\b")
```

```
[[1]]
[1] "3000"  "12345"
```

Greedy vs. Lazy

Greedy (por defecto): captura lo máximo posible

```r
x <- '<td>dato1</td><td>dato2</td>'

# Greedy: captura desde el primer < hasta el último >
str_extract(x, "<.+>")
```

```
[1] "<td>dato1</td><td>dato2</td>"
```

Lazy (con ?): captura lo mínimo posible

```r
# Lazy: se frena en el primer >
str_extract(x, "<.+?>")
```

```
[1] "<td>"
```

```r
# Extraer todas las etiquetas
str_extract_all(x, "<.+?>")
```

```
[[1]]
[1] "<td>"  "</td>" "<td>"  "</td>"
```

Se aplica a todos los cuantificadores: `*?`, `+?`, `??`, `{n,m}?`.

Alternancia: |

El operador | funciona como un «o» lógico entre patrones.

```
sectores <- c("Industria", "Agro", "Servicios",
              "Comercio", "Agropecuario")

str_detect(sectores, "Agro|Industria")
```

```
[1]  TRUE  TRUE FALSE FALSE  TRUE
```

```
# Extraer S.A. o S.R.L. (case insensitive)
empresas <- c("ACME S.A.", "globant s.r.l.",
              "Arcor", "YPF S.A.")

str_extract(empresas,
  regex("s\\.a\\.|s\\.r\\.l\\.",
        ignore_case = TRUE))
```

```
[1] "S.A."   "s.r.l." NA       "S.A."
```

Cuidado con la precedencia. `"gato|perro"` matchea `"gato"` o `"perro"`, no `"gat"` + (`"o"` o `"p"`) + `"erro"`. Use paréntesis si necesita acotar: `"gat(o|a)"`.

Grupos: ( )

Los paréntesis cumplen dos funciones: **agrupar** y **capturar**.

Agrupar para aplicar cuantificadores u OR

```r
x <- c("jajaja", "ja", "jejeje", "j")

# (ja)+ matchea "ja" repetido 1 o más veces
str_extract(x, "(ja)+")
```

```
[1] "jajaja" "ja"     NA       NA
```

```r
# gat(o|a|os|as): terminaciones múltiples
x <- c("gato", "gata", "gatos", "gatas", "gatito")
str_extract(x, "gat(o|a|os|as)")
```

```
[1] "gato"  "gata"  "gatos" "gatas" NA
```

Capturar para extraer subgrupos

```r
# devuelve las capturas en columnas
str_match("2025-01-15", "(\\d{4})-(\\d{2})-(\\d{2})")
```

```
     [,1]         [,2]   [,3] [,4]
[1,] "2025-01-15" "2025" "01" "15"
```

Grupos sin captura: (?:)

A veces necesitamos agrupar sin que el grupo se capture en el resultado.

```r
# Con grupo de captura: la columna extra aparece
str_match("2025-01", "(\\d{4})-(\\d{2})")
```

```
     [,1]      [,2]   [,3]
[1,] "2025-01" "2025" "01"
```

```r
# Con grupo sin captura (?:): no genera columna extra
str_match("2025-01", "(\\d{4})-(?:\\d{2})")
```

```
     [,1]      [,2]
[1,] "2025-01" "2025"
```

Es útil cuando queremos agrupar para aplicarle un cuantificador, sin que nos interese extraer ese grupo:

```r
# Matchear "http" o "https" sin capturar la "s"
str_extract("https://unl.edu.ar",
            "https?://\\S+")
```

```
[1] "https://unl.edu.ar"
```

Retrorreferencias (backreferences)

Se puede hacer referencia a lo que capturó un grupo anterior con \\1, \\2, etc.

```r
# Detectar palabras repetidas consecutivas
x <- c("la la casa", "el perro", "muy muy bien")
str_detect(x, "\\b(\\w+) \\1\\b")
```

```
[1]  TRUE FALSE  TRUE
```

```r
# Extraer la palabra repetida
str_match(x, "\\b(\\w+) \\1\\b")
```

```
     [,1]       [,2]
[1,] "la la"    "la"
[2,] NA         NA
[3,] "muy muy"  "muy"
```

```r
# En reemplazos: reorganizar "Apellido, Nombre"
str_replace("Perren, Jonatán", "(\\w+), (\\w+)",
            "\\2 \\1")
```

```
[1] "Jonatán Perren"
```

Lookahead y Lookbehind

Son «miradas» que verifican si un patrón está adelante o atrás, **sin consumir caracteres**.

Sintaxis	Descripción
`(?= ... )`	Lookahead positivo: lo que sigue es...
`(?! ... )`	Lookahead negativo: lo que sigue NO es...
`(?<= ... )`	Lookbehind positivo: lo anterior es...
`(?<! ... )`	Lookbehind negativo: lo anterior NO es...

Lookahead y Lookbehind: Ejemplos

Lookahead positivo (?=...)

```r
# Extraer el número que está antes de "kg"
str_extract("Peso: 150kg", "\\d+(?=kg)")
```

```
[1] "150"
```

Lookbehind positivo (? =...)

```r
# Extraer lo que está después de "Precio: $"
str_extract("Precio: $1500", "(?≤\\$)\\d+")
```

```
[1] "1500"
```

Lookahead negativo (?!...)

```r
# Números que NO están seguidos de "%"
str_extract_all("IPC: 5%, PBI: 150M",
                "\\d+(?!%)")
```

```
[[1]]
[1] "150"
```

Lookbehind negativo (? !...)

```r
# Números que NO están precedidos de "$"
str_extract_all("$100 y 200", "(?<!\\$)\\b\\d+")
```

```
[[1]]
[1] "200"
```

Ejemplo práctico: Validar formatos

CUIT argentino

Formato: XX−XXXXXXXX−X (con o sin guiones).

```r
cuits <- c("20-12345678-9", "20123456789",
           "12345", "30-99887766-0")

str_detect(cuits, "^\\d{2}-?\\d{8}-?\\d$")
```

```
[1]  TRUE  TRUE FALSE  TRUE
```

Correo electrónico (simplificado)

```r
emails <- c("juan@unl.edu.ar", "nope",
            "ana.garcia@gmail.com", "a@b")

str_detect(emails,
  "^[\\w.]+@[\\w]+\\.[a-z]{2,}(\\.[a-z]{2,})?$")
```

```
[1]  TRUE FALSE  TRUE FALSE
```

Patente argentina (formato nuevo)

```r
str_detect(c("AB123CD", "ABC123", "AB 123 CD"),
             "^[A-Z]{2}\\d{3}[A-Z]{2}$")
```

```
[1]  TRUE FALSE FALSE
```

Ejemplo práctico: Extraer datos de texto

Parsear números

```r
textos <- c("El IPC subió 3,5% en marzo",
            "Inflación interanual: 105,3%",
            "Sin variación")

str_extract(textos, "\\d+,\\d+(?=%)")
```

```
[1] "3,5"   "105,3" NA
```

Extraer fechas de texto libre

```r
x <- "Publicado el 15/03/2025 y revisado el 20/03/2025"

str_extract_all(x, "\\d{1,2}/\\d{2}/\\d{4}")
```

```
[[1]]
[1] "15/03/2025" "20/03/2025"
```

Separar nombre y valor

```r
x <- c("PBI: 150.3", "IPC: 5.2", "Desempleo: 7.1")

str_match(x, "(\\w+):\\s+([\\d.]+)")
```

```
     [,1]            [,2]        [,3]
[1,] "PBI: 150.3"    "PBI"       "150.3"
[2,] "IPC: 5.2"      "IPC"       "5.2"
[3,] "Desempleo: 7.1" "Desempleo" "7.1"
```

Modificar texto: str_replace y str_remove

Reemplazar texto

`str_replace()` cambia el **primer** patrón encontrado por cadena. `str_replace_all()` los cambia **todos**.

```
x ← c("manzana, pera, banana")

str_replace(x, ",", ";")
```

```
[1] "manzana; pera, banana"
```

```
str_replace_all(x, ",", ";")
```

```
[1] "manzana; pera; banana"
```

Eliminar texto

`str_remove()` equivale a `str_replace()` con un string vacío (`""`). También tiene versión `_all()`.

```
precios ← c("$1.200", "$4.500")

# Eliminamos el signo $ literal (escapado) de todos
str_remove_all(precios, "\\$")
```

```
[1] "1.200" "4.500"
```

Filtrar y separar el texto

str_subset: Filtrar un vector

Devuelve solo los elementos que **contienen** el patrón. Es equivalente a `x[str_detect(x, patron)]`.

```r
frutas <- c("manzana", "banana", "kiwi", "naranja")

# Solo las frutas que tienen "anja"
str_subset(frutas, "anja")
```

```
[1] "manzana" "naranja"
```

str_split: Separar cadenas

Divide una cadena en partes usando la regex como delimitador. Devuelve una **lista** por defecto.

```r
x <- "Ana, 35 años, Abogada"

# Separar por comas seguidas (opcionalmente) de espacios
str_split(x, ",\\s*")
```

```
[[1]]
[1] "Ana"     "35 años" "Abogada"
```

El súper poder: simplify = TRUE

Funciones que devuelven múltiples resultados en tamaño variable (como `str_extract_all` o `str_split`) retornan una **lista**. Si pasás `simplify = TRUE`, devuelven una **matriz**, lo cual es perfecto para meter en un data frame nuevo o un `mutate()`.

```r
x <- c("Juan-35", "Ana-28")
str_split(x, "-", simplify = TRUE)
```

```
     [,1]   [,2]
[1,] "Juan" "35"
[2,] "Ana"  "28"
```

```r
textos <- c("A: 10, B: 20", "C: 30")
str_extract_all(textos, "\\d+", simplify = TRUE)
```

```
     [,1] [,2]
[1,] "10" "20"
[2,] "30" ""
```

Si falta un valor (como en el segundo elemento del último ejemplo), la matriz se completa con cadenas vacías (`""`).

```r
library(dplyr)

datos <- tibble(
  texto = c(
    "Ventas: $1.250.000,50 (2024)",
    "Costos: $800.000,00 (2023)",
    "Ventas: $2.100.500,75 (2025)"
  )
)

datos |>
  mutate(
    concepto = str_extract(texto, "^\\w+"),
    monto = str_extract(texto, "(?<=\\$)[\\d.,]+") |>
        str_remove_all("\\.") |>
        str_replace(",", ".") |>
        as.numeric(),
    anio = str_extract(texto, "(?<=\\()\\d{4}(?=\\))") |>
        as.integer()
  )
```

```
# A tibble: 3 x 4
  texto                        concepto   monto  anio
1 Ventas: $1.250.000,50 (2024) Ventas   1250000. 2024
2 Costos: $800.000,00 (2023)   Costos    800000  2023
3 Ventas: $2.100.500,75 (2025) Ventas   2100501. 2025
```

Modificadores de patrón en stringr

Las funciones de `stringr` aceptan modificadores que cambian la interpretación.

Modificador	Descripción
`regex()`	Interpretación estándar como expresión regular (por defecto). Acepta `ignore_case`.
`fixed()`	Coincidencia literal, sin interpretar metacaracteres. Más rápido.
`coll()`	Comparación sensible a la localización (idioma).
`boundary()`	Coincide con límites (de palabra, oración, carácter).

```r
# Ignorar mayúsculas/minúsculas
str_detect("ECONOMÍA",
  regex("economía", ignore_case = TRUE))
```

```
[1] TRUE
```

```r
# Contar palabras con boundary
str_count("Tres simples palabras",
          boundary("word"))
```

```
[1] 3
```

Errores comunes

Olvidar duplicar la barra invertida

```r
str_extract("2025", "\d+")   # Error o no matchea
str_extract("2025", "\\d+")  # Correcto
```

No anclar al validar formatos

```r
# Sin anclas: "AB123CD extra" también matchea
str_detect("AB123CD extra", "[A-Z]{2}\\d{3}[A-Z]{2}")
```

```
[1] TRUE
```

```r
# Con anclas: solo matchea la cadena completa
str_detect("AB123CD extra",
           "^[A-Z]{2}\\d{3}[A-Z]{2}$")
```

```
[1] FALSE
```

Confundir str_extract con str_extract_all

`str_extract()` devuelve solo la **primera** coincidencia. Si necesitás todas, usá `str_extract_all()`.

Usar el punto sin escapar

Recordá que `"."` matchea **cualquier** carácter. Si buscás un punto literal, usá `"\\."` o `fixed(".")`.

Referencia rápida

Patrón	Descripción	Patrón	Descripción
`.`	Cualquier carácter	`( ... )`	Grupo de captura
`\d / \D`	Dígito / No dígito	`(?: ... )`	Grupo sin captura
`\w / \W`	Car. palabra / No	`a|b`	Alternancia
`\s / \S`	Espacio / No	`\1`	Retrorreferencia
`\b`	Límite de palabra	`(?= ... )`	Lookahead +
`^ / $`	Inicio / Final cadena	`(?! ... )`	Lookahead –
`[abc]`	Clase: a, b o c	`(?<= ... )`	Lookbehind +
`[^abc]`	Negación	`(?<! ... )`	Lookbehind –
`[a-z]`	Rango	`* + ?`	0+, 1+, 0/1 repet.
`{n} / {n,m}`	Exacto / Rango		

str_view: Tu mejor amigo para depurar

`str_view()` resalta visualmente las coincidencias. Usalo siempre para probar tus regex antes de aplicarlas.

```r
x <- c("Precio: $1.500,00", "Costo: $800,50")

str_view(x, "\\$[\\d.,]+")
```

```
[1] | Precio: <$1.500,00>
[2] | Costo: <$800,50>
```

```r
str_view(x, "\\d+")
```

```
[1] | Precio: $<1>.<500>,<00>
[2] | Costo: $<800>,<50>
```

```r
str_view(x, "(?≤\\$)[\\d.]+")
```

```
[1] | Precio: $<1.500>
[2] | Costo: $<800>
```